

Tópico 03

Modelação de Sistemas

Utilizar UML e Análise Orientada a Objetos para visualizar e estruturar a complexidade do software.

Princípios de Modelação

O mapa não é o território

01

Abstração

Esconder detalhes irrelevantes para focar no essencial. Um modelo deve ser mais simples que a realidade.

02

Comunicação

O objetivo principal de um modelo é comunicar ideias entre pessoas (técnicas e não-técnicas).

03

Múltiplas Vistas

Nenhum modelo único consegue capturar todo o sistema. Precisamos de vistas estáticas (estrutura) e dinâmicas (comportamento).

Espaço do Problema vs Solução

Onde estamos a trabalhar?

Espaço do Problema

O mundo real. As necessidades do utilizador. O "O Quê".

Conceitos: Cliente, Fatura, Email, Spam.



Espaço da Solução

O mundo técnico. A implementação. O "Como".

Conceitos: Base de Dados, API, Classe, Função, Algoritmo.

A modelação ajuda a fazer a ponte entre estes dois mundos.

Análise Orientada a Objetos (OOA)

Pensar em "Coisas" e não em "Funções"

O que é?

Uma técnica para modelar o sistema baseada nos objetos do mundo real (domínio do problema). Em vez de perguntar "o que o sistema faz?", perguntamos "quais são as entidades envolvidas?".

Como identificar objetos?

Procuramos **Substantivos** nas descrições dos requisitos. Ex: "O Cliente envia um Email para o Sistema." Objetos candidatos: Cliente, Email, Sistema.

CRC Cards

Diagramas de Classes

Casos de Uso

OOA: Cenários e Responsabilidades

Quem faz o quê?

Cenário: "Um email chega, é analisado e marcado como spam."

Objeto: Email

- Sabe o seu conteúdo (texto).
- Sabe o remetente.
- Não sabe se é spam ou não (inicialmente).

Objeto: Classificador

- Recebe um Email.
- Aplica regras ou modelo de IA.
- Devolve uma classificação.

Princípios OO na Modelação

Conceitos chave para bons modelos



Encapsulamento

Esconder a complexidade interna. O objeto expõe apenas o que é necessário (interface pública).



Herança

Reutilizar características comuns. Ex: "Email" e "SMS" podem herdar de "Mensagem".



Polimorfismo

Tratar objetos diferentes da mesma forma. Ex: Chamar ``enviar()`` funciona tanto para Email como para SMS.



Composição

Construir objetos complexos a partir de outros mais simples. Preferível à herança em muitos casos.

UML: A Linguagem Visual

Unified Modeling Language

Porquê UML?

É o padrão da indústria para visualizar, especificar, construir e documentar os artefactos de um sistema de software. Permite que todos falem a mesma língua visual.

Estruturais

Mostram a estrutura estática do sistema.
Ex: Diagrama de Classes, Componentes, Deployment.

Comportamentais

Mostram o comportamento dinâmico do sistema.
Ex: Casos de Uso, Sequência, Estados.

Diagrama de Casos de Uso

Definir as fronteiras e os objetivos de valor



Ator

Quem interage com o sistema. Pode ser uma pessoa (Cliente) ou outro sistema (API Externa).

Caso de Uso

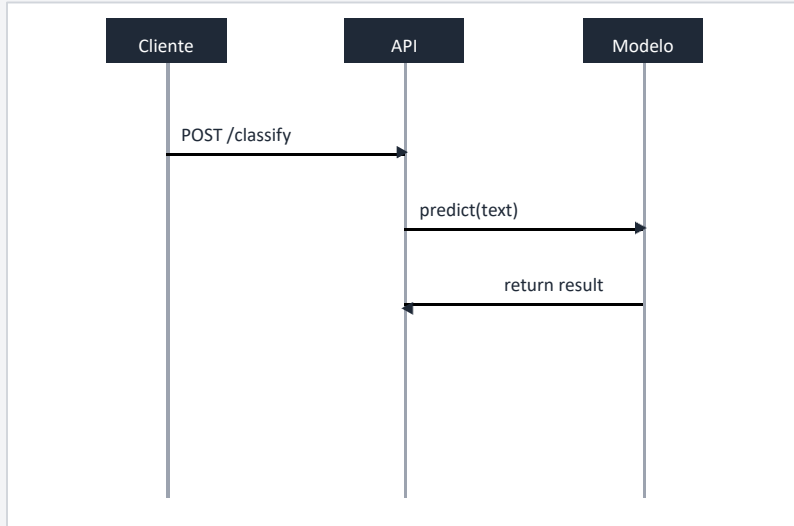
Um objetivo específico que traz valor. Deve começar com um verbo (ex: "Submeter", "Validar").

Fronteira

O que está dentro do sistema (nós construímos) vs. o que está fora (atores).

Diagrama de Sequência

Modelar interações e fluxo temporal



Linha de Vida

Representa um objeto ou componente participante na interação. O tempo flui de cima para baixo.

Mensagem Síncrona

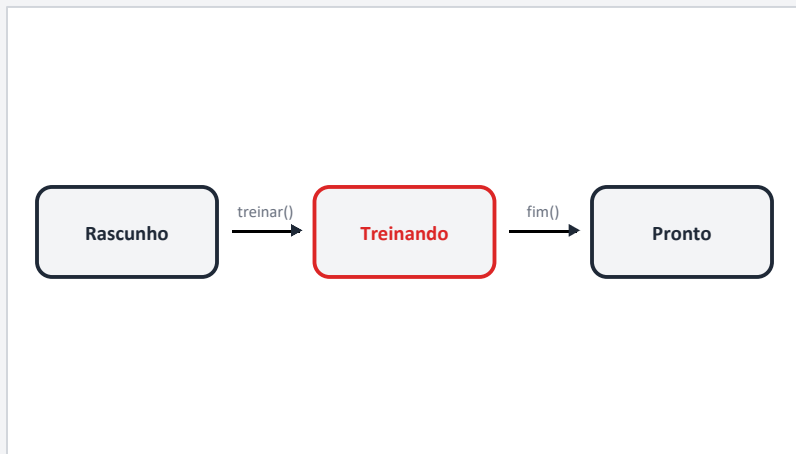
O remetente espera pela resposta (bloqueante). Comum em chamadas de função ou HTTP.

Foco de Controle

Mostra quando um objeto está ativo a processar algo (barras verticais na linha de vida).

Diagrama de Estados

Modelar o ciclo de vida e prevenir estados inválidos



Estado

Uma condição ou situação na vida de um objeto (ex: "A Treinar", "Aprovado").

Transição

A mudança de um estado para outro, geralmente desencadeada por um evento ou método.

Regra de Ouro

O diagrama define o que é **possível**. O sistema não pode saltar de "Rascunho" para "Pronto" sem passar pelo treino.